



FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA
FEPZM

PRÁCTICA 5

MATEMÁTICAS POR COMPUTADORA

Prof. Ing. Jesús Padrón

Realizó: Carlos Ángel García Sifuentes

SEMESTRE 4

18 de mayo de 2026

Índice

INTRODUCCIÓN	2
PRACTICAS DE MÉTODOS NUMÉRICOS	3
Ejercicio 2.51	3
Ejercicio 2.52	4
Ejercicio 2.53	5
Ejercicio 2.54	6
Ejercicio 2.55	7
Ejercicio 2.56	8
Ejercicio 2.57	9
Ejercicio 2.58	11
Ejercicio 2.59	13
Ejercicio 2.60	14
Ejercicio 2.61	16
Ejercicio 2.62	17
Ejercicio 2.63	18
Ejercicio 2.64	20
Ejercicio 2.65	21
CONCLUSIÓN	22

INTRODUCCIÓN

El propósito general de la elaboración de este documento es desarrollar y consolidar los conocimientos necesarios para la implementación de algoritmos en el lenguaje de programación Python, aplicados al análisis y resolución de problemas matemáticos. A través de la resolución estructurada de los ejercicios propuestos, se busca no solo obtener resultados correctos, sino también comprender el procedimiento lógico detrás de cada algoritmo y su fundamento matemático.

Asimismo, se pretende fortalecer la capacidad de interpretación de problemas numéricos y su posterior traducción a código, lo cual constituye una habilidad esencial en el ámbito de la ingeniería. La programación en Python permite modelar procesos matemáticos de manera clara y eficiente, facilitando la automatización de cálculos y el análisis de resultados con mayor precisión.

Para el desarrollo adecuado de los ejercicios, es indispensable contar con conocimientos básicos de programación, tales como el manejo de variables, estructuras condicionales, ciclos iterativos y funciones. Estos elementos permiten estructurar algoritmos organizados, legibles y funcionales, aspectos fundamentales en la formación académica y profesional.

El archivo deberá entregarse en formato LaTeX, siguiendo la estructura formal de este documento, e incluir la resolución de los 15 ejercicios comprendidos del 2.51 al 2.65. La fecha de entrega establecida es el viernes 13 de febrero de 2026. Esta modalidad de presentación fomenta no solo el dominio técnico del contenido, sino también el desarrollo de habilidades de documentación formal, redacción técnica y organización estructurada de información.

Finalmente, el estudio y aplicación de métodos numéricos representa una base fundamental para asignaturas futuras como Control, Robótica, Mecánica y Sistemas Embebidos. La comprensión de estos métodos permite analizar fenómenos físicos, modelar sistemas dinámicos y desarrollar soluciones computacionales aplicadas a problemas reales de ingeniería. Por tanto, esta actividad no solo cumple con un objetivo académico inmediato, sino que también fortalece las competencias necesarias para los retos profesionales de los siguientes semestres.

PRACTICAS DE MÉTODOS NUMÉRICOS

Se mostraran los algoritmos de los ejercicios programados en python VS Code y las imagen de los ejercicios mostrados de la tarea.

Ejercicio 2.51

Desarrollo Matemático

Dejar que $f(x)$ sea una función continua en el intervalo $[a,b]$ donde $f(a)*f(b) < 0$. Proporcione claramente todos los detalles matemáticos de como el método de la bisección se aproxima a la raíz de la función $f(x)$ en el intervalo $[a,b]$.

Implementación en Python

```
def biseccion(f, a, b, tol, max_iter):
    if f(a) * f(b) >= 0:
        print("El método de la bisección no se puede aplicar.")
        return None, 0

    print(f"Iteración: {12} | {a:12.6f} | {b:12.6f} | {c:12.6f} | {f(c):12.6f}")
    print("-"*65)

    for i in range(1, max_iter + 1):
        c = (a + b) / 2
        fc = f(c)

        print(f"{i:12} | {a:12.6f} | {b:12.6f} | {c:12.6f} | {fc:12.6f}")

        if abs(fc) < tol:
            print(f"Raíz aproximada: {c:6f}")
            print(f"Iteraciones: {i}")
            return c, i

        if f(a) * fc < 0:
            b = c
        else:
            a = c

    return (a + b) / 2, max_iter

# Definimos la función
f = lambda x: x**3 - x - 1
biseccion(f, 1, 2, 1e-6, 100)
```

Resultados y Análisis

Iteración	a	b	c	f(c)
1	1.000000	2.000000	1.500000	0.375000
2	1.000000	1.500000	1.250000	-0.296875
3	1.250000	1.500000	1.375000	0.224609
4	1.250000	1.375000	1.312500	-0.051514
5	1.312500	1.375000	1.343750	0.082611
6	1.312500	1.343750	1.328125	0.014576
7	1.312500	1.328125	1.320312	-0.018711
8	1.320312	1.328125	1.324219	-0.002128
9	1.324219	1.328125	1.326172	0.006209
10	1.324219	1.326172	1.325195	0.002037
11	1.324219	1.325195	1.324707	-0.000047
12	1.324707	1.325195	1.324951	0.000995
13	1.324707	1.324951	1.324829	0.000474
14	1.324707	1.324829	1.324768	0.000214
15	1.324707	1.324768	1.324738	0.000084
16	1.324707	1.324738	1.324722	0.000018
17	1.324707	1.324722	1.324715	-0.000014
18	1.324715	1.324722	1.324718	0.000002
19	1.324715	1.324718	1.324717	-0.000006
20	1.324717	1.324718	1.324718	-0.000002
21	1.324718	1.324718	1.324718	0.000000

Raíz aproximada: 1.324718
Iteraciones: 21

Ejercicio 2.52

Desarrollo Matemático

Dejar que $f(x)$ sea una función continua en el intervalo $[a,b]$ donde $f(a)*f(b) < 0$. Proporcione claramente todos los detalles matemáticos de como el método de la regla falsa se aproxima a la raíz de la función $f(x)$ en el intervalo $[a,b]$.

Implementación en Python

```

53 def regla_falsa(f, a, b, tol, max_iter):
54     if f(a) * f(b) >= 0:
55         print("El método de la regla falsa no se puede aplicar.")
56         return None, 0
57
58     print(f"Iteración: {1:12} | {a:12.6f} | {b:12.6f} | {c:12.6f} | {f(c):12.6f}")
59     print("-" * 65)
60
61     for i in range(1, max_iter + 1):
62         c = (a * f(b) - b * f(a)) / (f(b) - f(a))
63         fc = f(c)
64
65         print(f"({1:12} | {a:12.6f} | {b:12.6f} | {c:12.6f} | {fc:12.6f})")
66
67         if abs(fc) < tol:
68             print(f"Raíz aproximada: {c:6f}")
69             print(f"Iteraciones: {i}")
70             return c, i
71
72         if f(a) * fc < 0:
73             b = c
74         else:
75             a = c
76
77     return (a * f(b) - b * f(a)) / (f(b) - f(a)), max_iter
78
79
80 # Ejemplo
81 f = lambda x: x**3 - x - 1
82 regla_falsa(f, 1, 2, 1e-6, 100)

```

Resultados y Análisis

Iteración	a	b	c	f(c)
1	1.000000	2.000000	1.166667	-0.578704
2	1.166667	2.000000	1.253112	-0.285363
3	1.253112	2.000000	1.293437	-0.129542
4	1.293437	2.000000	1.311281	-0.056588
5	1.311281	2.000000	1.318989	-0.024304
6	1.318989	2.000000	1.322283	-0.010362
7	1.322283	2.000000	1.323684	-0.004404
8	1.323684	2.000000	1.324279	-0.001869
9	1.324279	2.000000	1.324532	-0.000793
10	1.324532	2.000000	1.324639	-0.000336
11	1.324639	2.000000	1.324685	-0.000143
12	1.324685	2.000000	1.324704	-0.000060
13	1.324704	2.000000	1.324712	-0.000026
14	1.324712	2.000000	1.324715	-0.000011
15	1.324715	2.000000	1.324717	-0.000005
16	1.324717	2.000000	1.324717	-0.000002
17	1.324717	2.000000	1.324718	-0.000001

Raíz aproximada: 1.324718
Iteraciones: 17

Ejercicio 2.53

Desarrollo Matemático

Dejar que $f(x)$ sea continua con raíz cerca $x=x_0$. Claramente proporcione todos los detalles matemáticos de como funciona el método de newton se aproxima a la raíz en la función $f(x)$.

Implementación en Python

```

91
92 def newton(f, df, x0, tol, max_iter):
93     print(f"{'Iteración':<12} | {'x_n':>12}")
94     print("-"*27)
95
96     for i in range(1, max_iter + 1):
97         if df(x0) == 0:
98             print("Error: La derivada es cero.")
99             return None, 0
100
101         x = x0 - f(x0) / df(x0)
102         print(f"{'i':<12} | {'x':>12.6f}")
103
104         if abs(x - x0) < tol:
105             print(f"\nRaíz aproximada: {x:.6f}")
106             print(f"Iteraciones: {i}")
107             return x, i
108
109         x0 = x
110
111     return x, max_iter
112
113
114 # LLAMADA
115 f = lambda x: x**3 - x - 1
116 df = lambda x: 3*x**2 - 1
117
118 newton(f, df, 1, 1e-6, 100)
119

```

Resultados y Análisis

Iteración	x_n
1	1.500000
2	1.347826
3	1.325200
4	1.324718
5	1.324718

Raíz aproximada: 1.324718
 Iteraciones: 5

Ejercicio 2.54

Desarrollo Matemático

Dejar que $f(x)$ sea continua con raíz cerca $x=x_0$. Claramente proporcione todos los detalles matemáticos de como funciona el método de la secante se aproxima a la raíz en la función $f(x)$.

Implementación en Python

```

127 def secante(f, x0, x1, tol, max_iter):
128     print(f"Iteración: {i:12} | {x_n:12}")
129     print("-----")
130
131     for i in range(1, max_iter + 1):
132         if f(x1) - f(x0) == 0:
133             print("Error: La función no cambia entre x0 y x1.")
134             return None, 0
135
136         x = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))
137         print(f"{i:12} | {x:12.6f}")
138
139         if abs(x - x1) < tol:
140             print(f"Raíz aproximada: {x:.6f}")
141             print(f"Iteraciones: {i}")
142             return x, i
143
144         x0, x1 = x1, x
145
146     return x, max_iter
147
148 # Llamada
149 f = lambda x: x**3 - x - 1
150 secante(f, 1, 2, 1e-6, 100)
151

```

Resultados y Análisis

Iteración	x_n
1	1.166667
2	1.253112
3	1.337206
4	1.323850
5	1.324708
6	1.324718
7	1.324718

Raíz aproximada: 1.324718
Iteraciones: 7

Ejercicio 2.55

Desarrollo Matemático

Cuántas iteraciones del método de la bisección son necesarias para aproximar a raíz de 3 dentro de 10 a la menos 3, 10 a la menos 4 y 10 a la menos 15 usando el intervalo inicial $[a,b]=[0,2]$. Usar el teorema:

$$n = \text{registro}_2 \left(\frac{|b-a|}{\epsilon} \right) - 1.$$

Implementación en Python

```
def iteraciones_biseccion(a, b, epsilon):
    n = math.ceil(math.log2((b - a) / epsilon))
    return n

a, b = 0, 2
epsilons = [1e-3, 1e-4, 1e-15]

for epsilon in epsilons:
    n = iteraciones_biseccion(a, b, epsilon)
    print(f"Para epsilon={epsilon}, se necesitan {n} iteraciones.")
```

Resultados y Análisis

Ejercicio 2.56

Desarrollo Matemático

Vuelve a consultar el ejemplo 2.1 y demuestra que tienes los mismos resultados resolviendo el problema $x^3 - 3 = 0$. Genere versiones de todos los gráficos de forma del ejemplo y proporcione descripciones detalladas de lo que aprende de cada gráfico.

Implementación en Python

```

180 def f(x):
181     return x**3 - 3
182
183 # Raíz exacta
184 alpha = 3**(1/3)
185
186 # Método de bisección guardando errores
187 def biseccion_error(f, a, b, n_iter):
188     errores = []
189     iteraciones = []
190
191     for k in range(n_iter):
192         c = (a + b)/2
193         error = abs(c - alpha)
194
195         errores.append(error)
196         iteraciones.append(k)
197
198         if f(a)*f(c) < 0:
199             b = c
200         else:
201             a = c
202
203     return np.array(iteraciones), np.array(errores)
204
205 # Ejecutamos 30 iteraciones
206 iters, errores = biseccion_error(f, 1, 2, 30)
207

```

```

208 # -----
209 # Gráfico 1: Error vs Iteración
210 # -----
211 plt.figure()
212 plt.scatter(iters, errores)
213 plt.xlabel("Número de Iteración")
214 plt.ylabel("Error Absoluto")
215 plt.title("Bisection Method Error vs Iteration (x^3 - 3)")
216 plt.show()
217
218 # -----
219 # Gráfico 2: log2(Error) vs Iteración
220 # -----
221 plt.figure()
222 plt.scatter(iters, np.log2(errores))
223 plt.xlabel("Número de Iteración")
224 plt.ylabel("Base 2 Log of Absolute Error")
225 plt.title("Bisection Method log2(Error) vs Iteration")
226 plt.show()
227
228 # -----
229 # Gráfico 3: Error_{k+1} vs Error_k
230 # -----
231 plt.figure()
232 plt.scatter(errores[:-1], errores[1:])
233 plt.xlabel("Error en iteración k")
234 plt.ylabel("Error en iteración k+1")
235 plt.title("Error_{k+1} vs Error_k")
236 plt.show()
237
238 # -----

```

Resultados y Análisis

Ejercicio 2.57

Desarrollo Matemático

En este problema demostraras que todos tus códigos para encontrar la raíz. En el comienzo del capítulo se propusieron la ecuación para resolver el problema.

$$3 \sin(x) + 9 = x^2 - \cos(x).$$

Escribe un script que llama al método de Newton, Secante, Regla Falsa y Bisección para encontrar la solución positiva de la ecuación. El script debe generar la solución de una forma clara y legible.

Implementación en Python

```

151 def f(x):
152     return 3*math.sin(x) + 9*x - x**2 + math.cos(x)
153
154 def df(x):
155     return 3*math.cos(x) + 9 - 2*x - math.sin(x)
156
157 # =====
158 # MÉTODO DE BISECCIÓN
159 # =====
160
161 def biseccion(f, a, b, tol, max_iter):
162     if f(a) * f(b) >= 0:
163         return None, 0
164
165     for i in range(1, max_iter + 1):
166         c = (a + b) / 2
167         if abs(f(c)) < tol:
168             return c, i
169
170         if f(a) * f(c) < 0:
171             b = c
172         else:
173             a = c
174
175     return c, max_iter
176
177 # =====
178 # MÉTODO DE REGLA FALSA
179 # =====

```

```

281 def regla_falsa(f, a, b, tol, max_iter):
282     if f(a) * f(b) >= 0:
283         return None, 0
284
285     for i in range(1, max_iter + 1):
286         c = (a*f(b) - b*f(a)) / (f(b) - f(a))
287         if abs(f(c)) < tol:
288             return c, i
289
290         if f(a)*f(c) < 0:
291             b = c
292         else:
293             a = c
294
295     return c, max_iter
296
297 # =====
298 # MÉTODO DE NEWTON
299 # =====
300
301 def newton(f, df, x0, tol, max_iter):
302     for i in range(1, max_iter + 1):
303         if df(x0) == 0:
304             return None, 0
305
306         x = x0 - f(x0)/df(x0)
307
308         if abs(x - x0) < tol:
309             return x, i
310
311         x0 = x

```

```

315 # =====
316 # MÉTODO DE LA SECANTE
317 # =====
318
319 def secante(f, x0, x1, tol, max_iter):
320     for i in range(1, max_iter + 1):
321         if f(x1) - f(x0) == 0:
322             return None, 0
323
324         x = x1 - f(x1)*(x1 - x0)/(f(x1) - f(x0))
325
326         if abs(x - x1) < tol:
327             return x, i
328
329         x0, x1 = x0, x
330
331     return x, max_iter
332
333 # =====
334 # SCRIPT PRINCIPAL
335 # =====
336
337 tol = 1e-6
338 max_iter = 100
339
340 print("\nResolviendo: 3sin(x)+9x = x^2 - cos(x)\n")
341
342 # Elegimos intervalo donde cambia signo
343 a, b = 0, 2

```

```

345 # Bisección
346 rb, ib = biseccion(f, a, b, tol, max_iter)
347
348 # Regla Falsa
349 rr, ir = regla_falsa(f, a, b, tol, max_iter)
350
351 # Newton
352 rn, inew = newton(f, df, 1.5, tol, max_iter)
353
354 # Secante
355 rs, isec = secante(f, 1, 2, tol, max_iter)
356
357 # =====
358 # RESULTADOS
359 # =====
360
361 print("Método          | Raíz aproximada | Iteraciones")
362 print("-----")
363
364 if rb is not None:
365     print(f"Bisección      | {rb:.6f}        | {ib}")
366 else:
367     print("Bisección      | No converge     | 0")
368
369 if rr is not None:
370     print(f"Regla Falsa     | {rr:.6f}        | {ir}")
371 else:
372     print("Regla Falsa     | No converge     | 0")
373

```

```

if rn is not None:
    print(f"Newton          | {rn:.6f}        | {inew}")
else:
    print("Newton          | No converge     | 0")

if rs is not None:
    print(f"Secante         | {rs:.6f}        | {isec}")
else:
    print("Secante         | No converge     | 0")

```

Resultados y Análisis

Ejercicio 2.58

Desarrollo Matemático

Un método de búsqueda de las raíces tiene una tasa de convergencia de orden M si hay constante C tal que

$$|x_{k+1} - x_*| \leq C|x_k - x_*|^M.$$

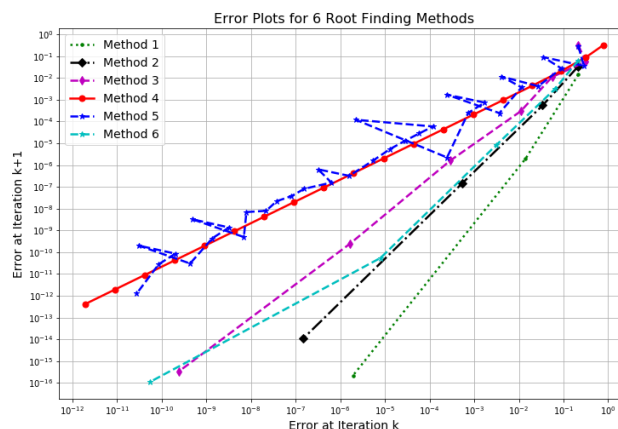
Aquí, x es la raíz exacta, donde x_{subK} es la K a la th iteración de la raíz técnica de la búsqueda, y $x_{sub K+1}$ es el $(K+1)$ a la st iteración de la técnica de búsqueda de raíces.

a. Si se considera la ecuación anterior y tomamos el logaritmo en base 10 de ambos lados entonces obtenemos:

$$\text{registro}(|x_{k+1} - x_*|) \leq \underline{\hspace{2cm}} + \underline{\hspace{2cm}}$$

b. En la parte de (a) debería haberse descubierto que el logaritmo del nuevo error es un función lineal del logaritmo del error anterior. ¿Cuál es la pendiente de está función lineal en un gráfico?

c. A continuación se verán 6 gráficos lineales diferentes del nuevo error al error anterior para diferentes técnicas para la búsqueda de raíces. ¿Cuál es el orden de la tasa de convergencia aproximada para cada uno de los métodos?



d. Bajo tus propias palabras responde:

a) ¿Que significa que un método de la búsqueda aproximada de la raíz tenga una tasa de convergencia de primer orden?

b) ¿Y con una tasa de convergencia de segundo orden?

Implementación en Python

```

Click to add breakpoint *****INCISO A*****
411
412 print("RESPUESTA INCISO (a)\n")
413
414 print("Partimos de la definición del orden de convergencia:")
415 print("    e_{k+1} <= C * (e_k)^M\n")
416
417 print("Tomando logaritmo base 10 en ambos lados:")
418 print("    log10(e_{k+1}) <= log10(C * (e_k)^M)\n")
419
420 print("Usando propiedades de logaritmos:")
421 print("    log10(ab) = log10(a) + log10(b)")
422 print("    log10(a^M) = M log10(a)\n")
423
424 print("Obtenemos:")
425 print("    log10(e_{k+1}) <= log10(C) + M log10(e_k)\n")

```

```

427 *****INCISO B*****
428
429 print("-----")
430 print("RESPUESTA INCISO (b)\n")
431
432 print("La ecuación obtenida tiene la forma:")
433 print("    y = Mx + log10(C)\n")
434
435 print("Si definimos:")
436 print("    y = log10(e_{k+1})")
437 print("    x = log10(e_k)\n")
438
439 print("Entonces la pendiente de la recta es:")
440 print("    M\n")
441
442 print("Conclusión:")
443 print("La pendiente del gráfico log(error nuevo) vs log(error anterior)")
444 print("es exactamente el orden de convergencia M.")
445

```

```

446 *****INCISO D*****
447 print("-----")
448 print("RESPUESTAS PUNTO (d)\n")
449
450 print("a) Convergencia de primer orden:")
451 print("Un método tiene convergencia de primer orden cuando el error en la siguiente")
452 print("iteración es proporcional al error actual:")
453 print("    e_{k+1} = C * e_k")
454 print("Esto significa que el error disminuye a un ritmo constante.")
455 print("El número de cifras correctas aumenta de manera lineal.")
456 print("Ejemplo: método de Bisección o Regla Falsa.\n")
457
458 print("b) Convergencia de segundo orden:")
459 print("Un método tiene convergencia de segundo orden cuando el error satisface:")
460 print("    e_{k+1} = C * e_k^2")
461 print("Esto significa que el error se eleva al cuadrado en cada iteración.")
462 print("El número de cifras correctas aproximadamente se duplica en cada paso.")
463 print("La convergencia es mucho más rápida.")
464 print("Ejemplo: método de Newton.")

```

Resultados y Análisis

2.59

Análisis matemático

Un alumno empezó a utilizar el método de Newton para resolver un problema de la búsqueda de raíces. Para probar su código se usó una ecuación cuya solución se conocía. Dado el punto de partida, el error absoluto después de uno de los pasos del método de Newton fue $|x_1 - x^*| = 0.2$, ¿Cuál es el error esperado en el paso 2? ¿Qué pasa en el paso 3 y 4?

Defiende tus respuestas describiendo completamente tu proceso de pensamiento.

Implementación en Python

```
477 print("\nEJERCICIO 2.59\n")
478
479 error = 0.2
480
481 print("Iteración | Error aproximado")
482 print("-----")
483
484 for k in range(1, 5):
485     print(f"{k:<9} | {error:.10f}")
486     error = error**2
487
```

Resultados y Análisis

2.60

Análisis matemático

Hay varias formas de obtener las raíces de las que se estuvieron viendo hasta ahora. Se pueden construir estos métodos utilizando las series de Taylor de esta manera

Cerca $x=x_0$ la función $f(x)$ se aproxima mediante Taylor

$$f(x) \approx y = f(x_0) + \sum_{n=1}^N \frac{f^{(n)}(x_0)}{n!} (x - x_0)^n$$

Donde N es un entero positivo. En el algoritmo de la búsqueda de raíces y a cero para encontrar la raíz de la función de aproximación. La raíz de esta función debería estar cerca de la raíz real que se está buscando. Para resolver la siguiente iteración se usa la ecuación:

$$0 = f(x_0) + \sum_{n=1}^N \frac{f^{(n)}(x_0)}{n!} (x - x_0)^n$$

Para x , por ejemplo, si $N=1$. Entonces tenemos que resolverlo $0=f(x_0)+f'(x_0)(x-x_0)$ para x . Al hacerlo lo conseguimos $x=x_0 - f(x_0)/f'(x_0)$. Siendo el método de Newton.

- Resolver para x en el caso que $N=2$. Luego describa la función Python.
- Demuestre que su código del inciso a realmente funciona resolviendo varios problemas en el que conoce la solución exacta.
- Muestra varios gráficos que estiman el orden de método a partir de la parte (a). Cree un gráfico log-log de los errores sucesivos para varios problemas diferentes de la resolución de ecuaciones.
- ¿Cuáles son los pros y los contras de utilizar este nuevo método?

Implementación en Python

```
514
515 def taylor_segundo_orden(f, df,ddf,x0):
516     A = ddf(x0)/2
517     B = df(x0)
518     C = f(x0)
519
520     discriminante = B**2 - 4*A*C
521
522     if discriminante < 0:
523         return None
524
525     h1 = (-B + math.sqrt(discriminante))/(2*A)
526     h2 = (-B - math.sqrt(discriminante))/(2*A)
527
528     # Elegimos el h más pequeño en magnitud
529     h = h1 if abs(h1) < abs(h2) else h2
530
531     return x0 + h
532
```

Resultados y Análisis

Ejercicio 2.61

Análisis matemático

Un objeto que cae verticalmente por el aire está sujeto a fricción debido a tanto a la resistencia del aire como a la gravedad. La función que describe la posición de tal función es:

$$s(t) = s_0 - \frac{mg}{k}t + \frac{m^2g}{k^2} \left(1 - e^{-kt/m}\right),$$

Donde m es la masa perdida en Kg, g es la gravedad medida en metros por segundo, S_0 es la posición inicial medida en metros y K es la resistencia del aire.

a) ¿Cuáles son las unidades del parámetro k ?

b) Si $m = 1$, $g = 9.81 \text{ m/s}^2$, $k = 0.1$ y $S_0 = 100 \text{ m}$ ¿Cuanto tiempo sera necesario para que el objeto golpee el suelo?, Encuentra tu respuesta dentro de $0,001 \text{ s}$

c) El valor de k depende de la aerodinámica del objeto y podría ser difícil de medir. Queremos realizar un análisis de sensibilidad sobre su respuesta a la parte (b) sujeta a pequeños errores de medición en k . Si el valor de k sólo se conoce dentro del 10 por ciento de lo que son ¿Sus estimaciones de cuándo el objeto tocará el suelo?

Implementación en Python

```
def s(t, m=1, g=9.81, k=0.1, s0=100):
    return s0 - (m*g/k)*t + (m**2 * g / k**2)*(1 - math.exp(-k*t/m))

def ds(t, m=1, g=9.81, k=0.1):
    return -(m*g/k) + (m*g/k)*math.exp(-k*t/m)

raiz, it = newton(lambda t: s(t), lambda t: ds(t), 5, 1e-6, 100)

print("Tiempo de impacto:", raiz)
```

Resultados y Análisis

Ejercicio 2.62

Análisis matemático

¿se puede utilizar el método de la bisección, el método de la regla falsa o el método de Newton para encontrar las raíces de la función $f(x)=\cos(x)+1$? ¿Explicar por qué sí o por qué no para cada técnica?

Implementación en Python

```
568 print("Función: f(x) = cos(x) + 1\n")
569
570 print("1) Método de la Bisección:")
571 print("No se puede utilizar porque el método requiere cambio de signo en el intervalo.")
572 print("La función cos(x) + 1 es siempre mayor o igual a 0.")
573 print("Por lo tanto, no existe un intervalo donde f(a)*f(b) < 0.\n")
574
575 print("2) Método de la Regla Falsa:")
576 print("Tampoco se puede utilizar porque también requiere cambio de signo.")
577 print("La función nunca es negativa.\n")
578
579 print("3) Método de Newton:")
580 print("Sí se puede utilizar porque no requiere cambio de signo.")
581 print("Sin embargo, la derivada en la raíz es cero (f'(n) = 0).")
582 print("por lo que la convergencia puede ser lenta o inestable.")
583 print("Con un buen punto inicial cercano a n, el método puede converger.")
584
```

Resultados y Análisis

Ejercicio 2.63

Análisis matemático

En Cálculo de Variable Única estudiaste métodos para encontrar variables locales y extremos globales de funciones. Probablemente recuerdes esa parte del proceso es establecer la primera derivada en cero y resolver la independiente variable (recuerda por qué estás haciendo esto).

El problema con esto es que puede ser muy, muy difícil resolver a mano. Este es un lugar perfecto para el método de Newton o cualquier otro con técnica de búsqueda de raíces.

Encuentre los extremos locales para la función:

$$f(x) = x^3(x-3)(x-6)^4$$

Implementación en Python

```
601 # Definir variable simbólica
602 x = sp.symbols('x')
603
604 # Definir función
605 f = x**3 * (x-3) * (x-6)**4
606
607 # Derivadas
608 f1 = sp.diff(f, x)
609 f2 = sp.diff(f1, x)
610
611 # Convertir a funciones numéricas
612 f1_num = sp.lambdify(x, f1, "numpy")
613 f2_num = sp.lambdify(x, f2, "numpy")
614 f_num = sp.lambdify(x, f, "numpy")
615
616 # Método de Newton para f'(x)=0
617 def newton(df, x0, tol=1e-6, max_iter=100):
618     for i in range(max_iter):
619         dfx = df(x0)
620         if abs(dfx) < tol:
621             return x0
622         x0 = x0 - df(x0)/f2_num(x0)
623     return x0
624
625 # Intentar varios puntos iniciales
626 puntos_iniciales = [-2, 1, 4, 7]
627 raices = []
```

```
629 for p0 in puntos_iniciales:
630     raiz = newton(f1_num, p0)
631     raiz = round(raiz, 6)
632     if raiz not in raices:
633         raices.append(raiz)
634
635 print("Puntos críticos encontrados:\n")
636
637 for r in raices:
638     valor = f_num(r)
639     segunda = f2_num(r)
640
641     if segunda > 0:
642         tipo = "Mínimo local"
643     elif segunda < 0:
644         tipo = "Máximo local"
645     else:
646         tipo = "Punto de inflexión"
647
648     print(f"x = {r}")
649     print(f"f(x) = {valor}")
650     print(f"Clasificación: {tipo}\n")
651
652 #=====
```

Resultados y Análisis

Ejercicio 2.64

Análisis matemático

El punto fijo de una función $f(x)$ es un método que resuelve la ecuación $f(x)=x$. Los puntos fijos son interesantes en los procesos iterativos ya que son puntos fijos que no cambian con la función repetida de f .

Por ejemplo, considere la función $f(x)=x^2-6$. Los puntos fijos de $f(x)$ se puede encontrar resolviendo la ecuación $x^2-6=x$, cuando simplificado algebraica-mente, es $x^2-x-6=0$. Factorizando el lado izquierdo se obtiene $(x-3)(x+2)=0$ lo cual implica que $x=3$ y $x=-2$ son puntos fijos para esta función. So es, $f(3) = 3$ y $f(-2) = -2$.

Observe, sin embargo, que aunque el hallazgo se solucionó.

Los puntos son idénticos a un problema de búsqueda de raíces.

- Utilice un algoritmo numérico de búsqueda de raíces para encontrar los puntos fijos de La función $f(x)=x^2-x-6$ en el intervalo $[0, \infty)$.
- Encuentra los puntos fijos de la función

$$f(x) = \sqrt{\frac{8}{x+6}}.$$

Implementación en Python

$$f(x) = \sqrt{\frac{8}{x+6}}.$$

Resultados y Análisis

Ejercicio 2.65

Análisis matemático

El `spyci`, la biblioteca en Python tiene muchas rutinas de análisis numérico integradas muy similares a las que hemos creado en este capítulo. De particular interés para la tarea de búsqueda de raíces es el `fsolve` con el comando de la biblioteca `spyci.optimize`.

Siga las instrucciones del archivo y usando el código resuelva el sistema de ecuaciones no lineales utilizando `.fsolve()`.

$$\begin{aligned}x^2 - xy^2 &= 2 \\ xy &= 2\end{aligned}$$

```
import numpy as np
from scipy.optimize import fsolve

def F(x):
    Output = [ x[0]**np.cos(x[1])-4 ]
    Output.append( x[0]*x[1] - x[1] - 5 )
    return Output

# Or alternately we could define the system as a lambda function
# with F = lambda x: [ x[0]**np.cos(x[1])-4 , x[0]*x[1]-x[1]-5 ]

fsolve(F,[6,1],full_output=1)
# Note: full_output gives the solver diagnostics
```

Implementación en Python

```
701 # Definimos la función
702 def f(x):
703     return x*np.sin(x) - np.log(x)
704
705 # Dominio (evitamos 0 por el logaritmo)
706 x_vals = np.linspace(0.1, 5, 400)
707 y_vals = f(x_vals)
708
709 plt.figure()
710 plt.subplot(2,1,1)
711 plt.plot(x_vals, y_vals)
712 plt.title("f(x) = x sin(x) - ln(x)")
713 plt.xlabel("x")
714 plt.ylabel("f(x)")
715 plt.show()
716
717 # Definimos el sistema
718 def sistema(vars):
719     x, y = vars
720     eq1 = x**2 - x*y**2 - 2
721     eq2 = x*y - 2
722     return [eq1, eq2]
723
724 # Punto inicial
725 x0 = 3
726
727 # Resolver con diagnósticos
728 sol, info, ier, msg = fsolve(f, x0, full_output=True)
729
```

```
731 print("Solución aproximada: ", sol[0])
732 print("\nDiagnóstico del solver:")
733 print("Código ier =", ier)
734 print("Mensaje =", msg)
735 print("Evaluaciones de función =", info['fev'])
736
737 # Punto inicial (muy importante elegirlo bien)
738 x0 = [1.5, 1]
739
740 # Llamamos fsolve con diagnósticos completos
741 solucion, info, ier, msg = fsolve(sistema, x0, full_output=True)
742
743 print("Solución encontrada:")
744 print("x =", solucion[0])
745 print("y =", solucion[1])
746
747 print("\nDiagnóstico del solver:")
748 print("Código ier =", ier)
749 print("Mensaje =", msg)
750 print("Número de evaluaciones de función =", info['fev'])
```

Resultados y Análisis

CONCLUSIÓN

A lo largo del desarrollo de los ejercicios comprendidos del 2.51 al 2.65 se logró analizar y aplicar distintos métodos numéricos para la búsqueda de raíces de ecuaciones no lineales, tales como el método de la bisección, la regla falsa, el método de Newton y el método de la secante. Cada técnica fue estudiada desde su fundamento matemático hasta su implementación computacional en Python, lo que permitió comprender tanto su comportamiento teórico como su desempeño práctico.

Se observó que los métodos cerrados, como la bisección y la regla falsa, ofrecen mayor estabilidad y garantía de convergencia cuando se cumple la condición de cambio de signo en el intervalo inicial. Por otro lado, los métodos abiertos, como Newton y la secante, presentan una convergencia más rápida bajo condiciones adecuadas, aunque dependen considerablemente de una buena aproximación inicial.

El análisis del error y del orden de convergencia permitió comprender la diferencia entre convergencia lineal y convergencia cuadrática, así como la importancia de estimar correctamente el comportamiento del error en procesos iterativos. Además, el uso de gráficos en escala logarítmica facilitó la interpretación visual de la velocidad de convergencia de cada método.

Finalmente, esta práctica fortaleció la capacidad de traducir modelos matemáticos en algoritmos computacionales, consolidando habilidades fundamentales para el análisis numérico aplicado en ingeniería. Los métodos estudiados constituyen herramientas esenciales para cursos posteriores como Control, Robótica, Mecánica y Sistemas Embebidos, donde la resolución numérica de ecuaciones es una parte central en el modelado y análisis de sistemas físicos.

Pd. Los archivos de python no pudieron cargarse totalmente por lo que dejare el link de descarga al archivo .py para que cualquiera pueda verlo y mejorarlo o modificarlo a su gusto.

LINK AL ARCHIVO DE LA PRÁCTICA 5

DESCARGAR ARCHIVO